重みのメモリ配置と推論速度 調査報告

- 実験日: 2026 年 10 月 13 日（追加実験: 10 月 15 日、4.4・4.5 節）
- 担当: Naoya Sakurada
- ブランチ: `dev/sakurada`（`dev/cpu-profiler` から分岐）
- 機材: 学科 PC EE00025（Intel Core Ultra 5 225U、DDR5-5600 16GB×2、L2 合計 10MB、L3 12MB）
- モデル: `stories15M.bin`（重み 60.8MB）

要約

1. `run.c` に、重みをメモリにどう置くかを切り替える `-w` オプションを試作した（位置揃え・ヒュージページ）。出力は全モードで完全に一致する。
2. **ネイティブ Linux での効果は、1 スレッドで約 8%（10 回中 10 回で高速）、4 スレッドで約 2%。** 大きな効果ではない。
3. WSL では「重みが物理メモリ上でバラバラに置かれると、最大で約 3 割遅くなる」という現象が見つかった。**ネイティブ Linux ではこの現象は起きなかった。** 原因は仮想化（Windows 側の 2 段目の住所変換）と考えられる。同じ PC での比較により、**WSL ではヒュージページにしても TLB が 2MB 単位で効かない**ことも確かめた（ネイティブでは効く）。
4. **ネイティブ Linux の電源プロファイルで、1 スレッドの速さが 2.2 倍変わる。** 標準の「バランス」では 1 スレッドのメモリ読み出しが約 10 GB/s に抑えられ、204 tok/s。「パフォーマンス」にすると 26.5 GB/s・447 tok/s になる（4 スレッドでは 1.1 倍の差）。以前の「ネイティブの方が WSL より遅い」の原因はこれだった。**班の測定は、ネイティブ Linux で電源プロファイルを「パフォーマンス」にそろえて取るべき。**
5. WSL でスレッド数を指定しないと 14 スレッドになり、速さが約 1/170（4.8 tok/s）に落ちる。WSL で測るときは必ず `OMP_NUM_THREADS` を指定すること。

---

1. 背景と目的

`run.c` は 1 トークンごとに重み 60.8MB を全部メモリから読む。L3 キャッシュ（12MB）には収まらないので、`-Ofast -march=native` でビルドすると、メモリの読み出しが速さの上限を決めやすい（WSL での測定では、1 スレッドで読み出し限界の約 8 割に達していた）。

そこで、**計算結果を変えずに、メモリの読み方だけを改善できないか** を調べた。

---

2. 試作：`-w` オプション

元の `run.c` は、重みファイルを `mmap` で直接使う。このとき、重みの先頭はファイルの 28 バイト目にあり（`Config` のすぐ後ろ）、64 バイト単位にそろっていない。

`-w`	重みの置き方	前のモードと比べると分かること
0	元のまま、ファイルを `mmap` する（先頭が 28 バイトずれている）	比較の基準
1	普通のメモリにコピーする（ずれは 28 バイトのまま、4KB ページ）	ファイルを `mmap` するか、コピーするか
2	先頭を 2MB 境界にそろえてコピーする（4KB ページ）	位置揃えの効果
3	2 に加えて `MADV_HUGEPAGE`（2MB のヒュージページ）	ヒュージページの効果

- 起動時に `weights: mode=… load_ms=… offset_mod64=… anon_huge_kb=…` を表示し、狙いどおりの置き方になっているかを確認できる。
- コピーするモードは、起動が約 20〜25ms 遅くなる。
- ベンチマーク用スクリプト `bench_weights_layout.sh` と集計用 `analyze_weights_layout.py` も追加した。モードとスレッド数の組み合わせを毎周ランダムな順番で実行し、全試行の出力が同じかを SHA-256 で確かめる。

---

3. 測定環境

	WSL2（Windows 上）	ネイティブ Linux
OS	Ubuntu（カーネル 6.6.87.2-microsoft-standard-WSL2）	Ubuntu 24.04.3（カーネル 6.17.0）
電源	AC 電源、Windows の電源プランは「バランス」	AC 電源、電源プロファイル「バランス」（`powersave` / `balance_performance`）。4.5 節のみ 3 種類を比較
CPU の見え方	仮想化されている（「2 スレッド × 7 コア」に見え、P/E コアの区別がない）	実物どおり（P コア 0〜3、E コア 4〜11、LP E コア 12〜13）
1 スレッドでの読み出し速度	17〜19 GB/s	約 10 GB/s（バランス）/ 26.5 GB/s（パフォーマンス）
ヒュージページの設定	`madvise`	`madvise`

注意: WSL の CPU 番号は物理的なコアと対応していない。`taskset` でコアを固定しても、P コア・E コアを選んだことにはならない（コアごとに測っても 2 つのグループに分かれず、同じ番号でも周によって速さが変わった）。

---

4. 結果

4.1 WSL での `-w` の比較（1 秒あたりのトークン数の中央値、15 周）

スレッド数	mmap	copy	aligned	aligned+huge	元に対する合計
1	362.3	282.2	290.8	**400.4**	1.105 倍（15/15）
4	547.0	531.2	523.7	**565.2**	1.033 倍（14/15）
8	786.5	751.7	756.8	770.0	0.979 倍（6/15、差なし）

1 スレッドで 1 つずつ変えたときの効果（同じ周どうしで比べた値）：

比べたもの	倍率	勝ち数
普通のメモリにコピー（copy と mmap）	**0.779**	0/15
位置揃え（aligned と copy）	1.030	15/15
ヒュージページ（aligned+huge と aligned）	**1.377**	15/15

謎: 普通のメモリにコピーしただけで、約 2 割遅くなった。

4.2 WSL での謎の調査：遅いのは「バラバラに置かれたページ」

(1) mmap か、普通のメモリかは関係なかった（`run.c` 1 スレッド、20 周）

重みの置き場所	元のファイルに対する倍率	勝ち数
A. 元のファイルを mmap	1.000（274.2 tok/s）	—
B. 作ったばかりのコピーを mmap	**0.896**	1/20
C. tmpfs（`/dev/shm`）上のコピーを mmap	1.024	17/20
D. 普通のメモリ・4KB ページ	**0.910**	1/20
E. 普通のメモリ・ヒュージページ	1.072	18/20

同じ mmap でも、作ったばかりのコピー（B）は普通のメモリ（D）と同じくらい遅い。

(2) ページキャッシュを作り直すと、速い・遅いが入れ替わった

A と B のページキャッシュを捨て（`dd iflag=nocache`）、A → B の順に読み直した。すると A は 274.5 → 214.8 tok/s に落ち、B は 239.4 → 262.0 tok/s に上がった。速さを決めているのは、ファイルではなく **ページが載った物理メモリ** である。

(3) 遅いのは住所の変換ではなく、データの読み出し（同じページ配置のまま測定）

	連続読み出し	`run.c` と同じ行列計算	住所を調べる時間（ランダムな順で読んだ場合）
遅い方	15.6 GB/s	14.1 GB/s	5.2 ns
速い方	17.1 GB/s	17.4 GB/s（**1.23 倍**）	8.2 ns

行列計算の速さの比（1.23 倍）は、`run.c` の速さの比（1.22 倍）と一致した。

(4) 先に読み込んだファイルが、必ず遅いメモリを引き当てる

読み直す順番	元のファイル	コピー
元のファイルを先に（6 回）	13.8〜14.3 GB/s（遅い）	16.4〜16.9 GB/s
コピーを先に（3 回）	15.7〜16.0 GB/s	13.3〜13.5 GB/s（遅い）
関係ない 150MB のファイルを先に（3 回）	**18.2〜18.9 GB/s**	**18.2〜18.9 GB/s**

（数値は `run.c` と同じ行列計算の速さ）

Linux は、空きメモリのうち **細切れの空き（あちこちに散らばっている）を先に使う**。この時点で細切れの空きは約 149MB あった（`/proc/buddyinfo`）。そのため、最初に確保した 60MB は物理的にバラバラなページに載

る。150MB を先に読ませて細切れの空きを使い切らせると、両方ともひと続きのページに載り、速くなった。

(5) 同じページのまま、並べ方だけを変えた（`memlab/chunkshuffle.c`）

64MB 分のページの物理的な位置を固定し、並べ方だけを変えて読む速さを比べた。

並べ方	行列計算の速さ（そのままの順番 = 1.00）
4KB ずつバラバラ	**0.70**
16KB ずつ	0.72
64KB ずつ	0.91
256KB ずつ	0.97
1MB ずつ	0.99

4 回とも同じ傾向で、4KB ずつバラバラにしたときは 0.70〜0.82 だった。「どのページか」ではなく「並び方」で速さが決まる。

4.3 ネイティブ Linux での確認

同じ実験を、同じ PC をネイティブ Linux で起動して行った（`run_native_all.sh`。1 スレッドの実験は P コアの CPU 2 に固定）。

実験	WSL	ネイティブ Linux
4KB ずつバラバラに並べ替え（行列計算）	0.70〜0.82 倍	**0.98〜1.00 倍**
置き場所 5 種類（4KB ページ同士の比較）	0.90 倍前後	**1.00〜1.015 倍**
読み込む順番の入れ替え	先に読んだ方が約 2 割遅い	**差なし**（12.8〜13.1 GB/s）
ヒュージページ（E）	1.072 倍	1.074 倍（15/20）

ネイティブ Linux での `-w` の比較（10 周）：

スレッド数	mmap	copy	aligned	aligned+huge	元に対する合計
1	207.1	208.7	214.9	**223.6**	**1.079 倍（10/10）**
4	594.1	566.7	590.9	606.3	1.021 倍（6/10）

（2 スレッドは、同じ条件でも 23〜122% もぶれたので、判断に使っていない。）

4.4 TLB がカバーできる範囲（訂正あり）

ページ 1 枚につき 1 回だけ、ランダムな順番で読む測定（`memlab/tlbreach.c`）。

- **最初の版には欠陥があった。** 読む位置をページ番号から単純に決めていたため、物理的にひと続きのヒュージページでは、読むデータがキャッシュの同じ 64 か所に集中していた。その結果、ネイティブ Linux で「ヒュージページの方が遅い」という誤った結果が出た。読む位置をハッシュで散らす修正版に差し替えた。
- **修正版の WSL での結果**: 4KB ページは 0.73ns → 2.9〜3.4ns、ヒュージページは 0.73ns → 2.4〜2.9ns と、どちらも 8〜16MB を超えたあたりで遅くなった。
- **修正版のネイティブ Linux での結果**（10 月 15 日、`power_tlb_check.sh`）：

読む範囲	4KB（P コア）	ヒュージページ（P コア）	4KB（E コア）	ヒュージページ（E コア）
4MB まで	0.74〜0.99 ns	0.71〜0.92 ns	約 1.46 ns	約 1.35 ns
16MB	1.8 ns	**0.69 ns**	9.3 ns	**1.30 ns**
58MB	2.3〜3.5 ns	**1.0〜1.3 ns**	10.6 ns	**1.35 ns**

ネイティブ Linux では、ヒュージページはほとんど遅くならない（2MB のページ 1 枚を TLB の 1 項目で覚えられている）。一方 WSL では、ヒュージページも 4KB とほぼ同じように遅くなった。**同じ PC・同じプログラムでの比較から、WSL では Linux 側でヒュージページにしても、Windows 側（2 段目）の住所変換のせいで TLB が 2MB 単位で効かないことが確かめられた。**

4.5 電源プロファイルと速さ（ネイティブ Linux、10 月 15 日）

`powerprofilesctl` の 3 つのプロファイルを、順番を毎周入れ替えながら 6 周ずつ測った（`power_tlb_check.sh`）。1 スレッドは P コアの CPU 2 に固定し、4 スレッドは P コア（CPU 0〜3）で実行した。値は中央値で、各周のぶれは数 % 以内。

	省電力	バランス（標準）	パフォーマンス
メモリの読み出し速度（1 スレッド）	10.8 GB/s	10.6 GB/s	**26.5 GB/s（2.50 倍）**
`run.c` 1 スレッド（`-w 0`）	188.5 tok/s	203.5 tok/s	**447.4 tok/s（2.20 倍）**
`run.c` 1 スレッド（`-w 3`）	199.3	220.8	469.2
`run.c` P コア 4 スレッド（`-w 0`）	280.3	571.8	**630.5（1.10 倍）**
`run.c` P コア 4 スレッド（`-w 3`）	293.5	596.5	660.7
1→4 スレッドで何倍速くなるか（`-w 0`）	1.49 倍	**2.81 倍**	**1.41 倍**

（倍率は、断りがなければバランスに対するもの）

- CPU の基本設定（`scaling_governor`）は 3 つとも `powersave` で、変わるのは `energy_performance_preference`（`power` / `balance_performance` / `performance`）と `platform_profile`（`low-power` / `balanced` / `performance`）。
- `-w 3`（ヒュージページ）は、どのプロファイルでも速かった。1 スレッドで 1.05〜1.09 倍（18 回中 17 回で勝ち）、4 スレッドで 1.04〜1.05 倍（18 回中 18 回で勝ち）。

---

5. 考察

5.1 「バラバラだと遅い」の原因

- ネイティブ Linux では、物理アドレスがそのまま本物のメモリの位置になる。そこで本当にページの並びをバラバラにしても、遅くならなかった。つまり、**メモリの部品側の理由（DRAM の行の開け閉めや、区画・通路への振り分け）は、この PC では原因ではない。**
- WSL にだけあるのは、Windows（Hyper-V）側の **2 段目の住所変換** である。Linux 側のページがバラバラに並んでいると、2 段目の変換表も毎回離れた場所を引くことになり、変換の手間が増える、と考えられる。

- 4.2 (3) の「住所を調べる時間」は、ページをランダムな順番で読んで測ったものなので、順番に読むときに表の近い部分を続けて使える効果は捉えられていない。5.1 の結論とは矛盾しない。
- ただし、2 段目の変換表をたどる回数を直接数えたわけではない。WSL ではカウンタが使えず、学科 PC では perf が許可されていなかった（`perf_event_paranoid=4`）。
- それでも、4.4 節の比較（ネイティブではヒュージページで TLB が 2MB 単位で効くのに、WSL では効かない）は、WSL の 2 段目の住所変換が細かい単位で行われていることを直接示しており、この結論を強く支持する。

5.2 ヒュージページの効果

- ネイティブ Linux では 1 スレッドで 4〜8% 速くなる。TLB に入りきらない分の、住所を調べる手間が減るためと考えられる。
- スレッドが増えると効果は小さくなる。各コアが読む範囲が小さくなり、4KB ページのままでも TLB に入りやすくなるためと考えられる。
- WSL で効果が大きく見えたのは、ヒュージページが必ず 2MB のまとまった場所に置かれ、WSL 特有の「バラバラの不利」も避けられたため。

5.3 測定条件についての注意

- ネイティブ Linux の方が、1 スレッドでの読み出しも推論も遅かった（約 10 GB/s、207 tok/s）。**原因は電源プロファイルだった**（4.5 節）。「パフォーマンス」にすると 26.5 GB/s・447 tok/s になり、WSL（274〜362 tok/s）より速い。班のメモにある「P コア 1 つで 4.8ms/トークン」も、今回の「バランス」での値（207 tok/s = 4.8ms）と一致するので、班のこれまでの測定は「バランス」で取られていたと考えられる。
- 「バランス」では、1 スレッドのときにメモリの読み出しが抑えられ、スレッドを増やすと抑えが外れるように見える（1 → 4 スレッドで 2.81 倍。「パフォーマンス」では 1.41 倍）。CPU が「忙しくない」と判断している間は、メモリまわりの周波数を下げているのではないかと考えている（推測。メモリまわりの周波数は直接測っていない）。そのため、**「バランス」で測ったスレッド数の効果には、電源管理の効果が混ざっている**。
- ネイティブ Linux の方が読み出し速度が低かったため、「バラバラだと遅い」の差が小さく見えた可能性はある。ただ、WSL をバッテリー駆動にして読み出し速度が同じくらい（約 11 GB/s）に落ちたときも、差は残っていた（約 0.92〜0.95 倍）。そのため、ネイティブで差が消えたのは条件の違いのせいではない、と判断している。

---

6. 班の実験への提案

1. **ベンチマークはネイティブ Linux で取る。** WSL では CPU 番号が物理コアと対応しないうえ、メモリの置かれ方しだいで最大 3 割ぶれる。
2. **電源プロファイルを「パフォーマンス」にそろえて測る。** 測定の前に `powerprofilesctl set performance` を実行し（sudo は不要）、終わったら `powerprofilesctl set balanced` で戻す。あわせて `powerprofilesctl get` の結果と、AC 電源につながっているかを記録する。
3. **これまで「バランス」で出したスレッド配置の結論は、「パフォーマンス」で測り直す。** 「バランス」では、スレッドを増やしたときの速さの伸びに、電源管理の抑えが外れる効果が混ざっている（4.5 節）。
4. **比べたい構成は、同じ時間帯に交互に測る**（今の班のスクリプトと同じやり方）。

5. `-w 3`（ヒュージページ）は、どの電源設定でも 4〜9% 速い。ただ、班の主題であるスレッド配置の最適化と比べると、効果は小さい。

---

7. 未解決・次の実験

項目	方法
班のスレッド配置実験の測り直し	電源プロファイルを「パフォーマンス」にして、`bench_affinity.sh` などを測り直す
E コア・LP E コアでの電源プロファイルの影響	今回は P コアだけで測った。`PCPU=4 bash power_tlb_check.sh` のように E コアでも測る
「バランス」で読み出しが抑えられる仕組み	メモリまわりの周波数を測る（perf の uncore カウンタなど。perf の許可が必要）
2 段目の変換が原因であることの直接確認	WSL ではカウンタが使えないので難しい。Windows 側で Hyper-V のカウンタを見られれば確認できる可能性がある
perf による TLB ミスの回数	学科 PC で `perf_event_paranoid` を下げてもらう必要がある

---

8. 再現方法

ブランチ `dev/sakurada` のファイル:

ファイル	内容
`run.c`	`-w` オプションを追加した版
`bench_weights_layout.sh` / `analyze_weights_layout.py`	`-w 0〜3` のベンチマークと集計
`run_native_all.sh`	ネイティブ Linux 用の一括実験（E1〜E6）。結果を `results_<ホスト名>_<日時>.tar.gz` にまとめる
`benchmark_weights_layout/`	WSL での `-w` の比較結果（4.1 節）
`memlab/tlbreach.c`	TLB がカバーする範囲（修正版）
`memlab/chunkshuffle.c`	同じページを並べ替える実験
`memlab/probe.c`	ファイルの今のページ配置のまま、読み出し速度と住所を調べる時間を測る
`memlab/memlab.c` / `memlab/resident.c`	メモリの種類ごとの比較 / ページキャッシュに載っている割合
`memlab/native_EE00025_20261013/`	ネイティブ Linux の結果一式（4.3 節）
`power_tlb_check.sh`	ネイティブ Linux 用の追加実験（修正版 TLB と電源プロファイル）。必要なファイルは GitHub から取得する
`memlab/native_power_EE00025_20261015/`	追加実験の結果（4.4・4.5 節）
`memlab/*.txt`	WSL での各実験の生データ（4.2・4.4 節）

`-w` の比較（リポジトリの直下で実行）：

```
REPEATS=15 ./bench_weights_layout.sh
```

ネイティブ Linux での一括実験（リポジトリの直下で実行。約 5 分、AC 電源が必要、sudo は不要）：

```
./run_native_all.sh
```

モデルは `stories15M.bin` があればそれを、なければ `build/stories15M.bin` を使う。1 スレッドの実験は CPU 2 に固定する（`PCPU=0 ./run_native_all.sh` のように変更できる）。

追加実験（clone 不要。ネイティブ Linux のターミナルで 1 行実行するだけ。約 3 分、終わると電源プロファイルを元に戻す）：

```
curl -fsSL https://raw.githubusercontent.com/doss-eeic/2026-07-llama2c/dev/sakurada/power_tlb_check.sh -o power_tlb_check.sh && bash power_tlb_check.sh
```

付録：用語

用語	意味
ページ	OS がメモリを管理する単位。普通は 4KB
ヒュージページ（THP）	2MB の大きなページ。`madvise(MADV_HUGEPAGE)` で頼むと使われる
ページキャッシュ	OS が、読んだファイルの中身をメモリに置いておく場所
mmap	ファイルの中身（ページキャッシュ）を、コピーせずにプログラムのメモリとして直接見せる仕組み
TLB	CPU が「仮の住所 → 本当の住所」の変換結果を覚えておく小さな表。容量を超えると、メモリ上の変換表をたどり直す必要がある
2 段目の住所変換	仮想マシン（WSL2）では、Linux の変換のあとに Windows（Hyper-V）側でもう一度住所を変換する
tok/s	1 秒あたりに生成できるトークン数。大きいほど速い
電源プロファイル	Ubuntu の「省電力 / バランス / パフォーマンス」の設定。`powerprofilesctl get` / `set` で確認・変更できる